LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Advanced Topics in Parallel Computing

Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

The C++ Memory Model and Atomic Operations

Part 1



Recap

- We've discussed memory consistency models offered by shared memory hardware
  - Strong (SC, TSO) and weak models, difference is in the **amount of reordering permitted** in memory access operations (loads/stores)
  - Almost all existing hardware allows **some** reorderings to happen
  - The term **reordering** here refers to the view from other threads, the thread issuing the operations still sees things in order
  - For useful multithreaded programming we need to be able to **limit these reorderings**
  - This can be done using **synchronizing operations** (sync. loads/stores or memory fences)
  - We can specify the reordering restrictions in pseudo-code using the "double bar" notation (e.g., ||RW)

Plan for Today

- Discuss how a high-level language deals with memory consistency
  - We'll focus on C++, where a lot of effort went into the specification of a memory model with C++11
  - (Java and Rust are other important languages with similar challenges)
- The C++ Memory model
  - Structural aspects
  - Concurrency aspects
- Race conditions and undefined behavior
  - What happens if synchronization is missing
- Atomic operations and atomic types
  - The building blocks of all synchronization functionality in C++
- Happens-before and synchronizes-with relationship
  - Establishing order between threads

} Part 2

The Need for a C++ Memory Model

- Major changes and enhancements with C++ 11
  - Support for concurrency (i.e., threads) in the language and library
  - Requires a memory model

- Hans Boehm
  - "Threads Cannot be Implemented as a Library"
    ACM Sigplan Notices 40.6 (2005): 261-268.

- Java Memory model was redefined at around the same time
  - "The Java memory model
    is fatally flawed"
    William Pugh. Concurrency:
    Practice and Experience
    12.6 (2000): 445-455.



Sarita Adve, University of Illinois at Urbana-Champaign



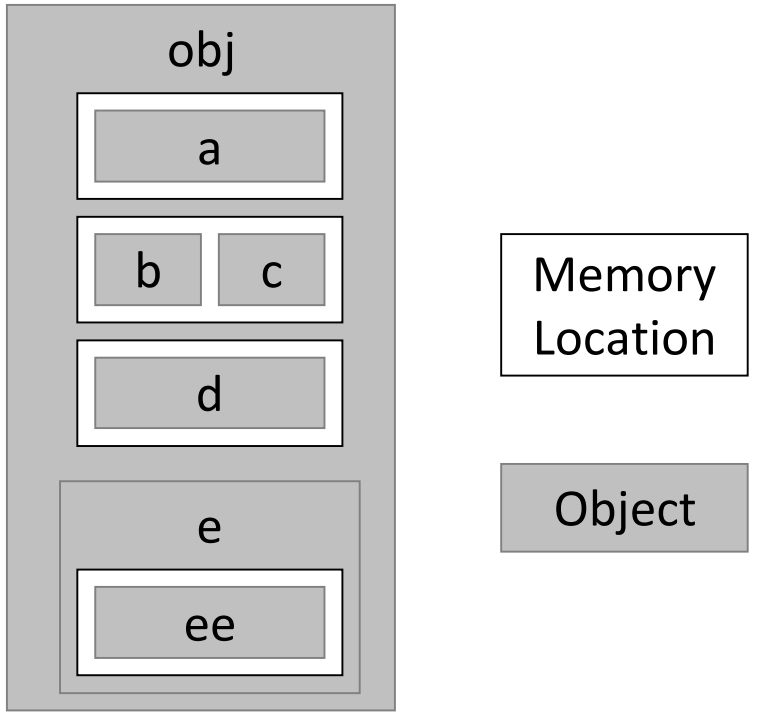
Hans Boehm, Google

Structural Aspects of the Memory Model (1)

- C++ organizes data in terms of **objects** in memory
  - An object is defined as a **region of storage**
  - Every variable corresponds to a memory object
  - Each memory object is stored in one or more **memory locations**
  - Variables of fundamental types (int/char/float/double/pointer/…) occupy **exactly one** memory location
  - Adjacent bitfields are **part of the same memory location**

```
struct S {
    char a;      // mem. loc. #1
    int b : 5;   // mem. loc. #2
    int c : 11, // mem. loc. #2 (cont.)
          : 0,
        d : 8;   // mem. loc. #3
    struct SS {
        int ee : 8; // mem. loc. #4
    } e;
} obj;
```

Example from www.cppreference.com, visualization inspired by Anthony Williams *"C++ Concurrency in Action"*



Structural Aspects of the Memory Model (2)

- The notion of **memory location** (not variable, object) is the basis for the definition of a data race in C++

- Definitions
  - Two accesses to the **same memory location** by **two different threads**, where at least one is a write operation, are said to be **conflicting**
  - Two conflicting accesses constitute a **data race**, unless
    - **Both** accesses are performed through **atomic operations**, or
    - One conflicting access **happens before** the other
  - A situation where conflicting access appear is also called a **race condition**

- Consequences of a data race in C++:
  - Incorrect results (e.g., lost updates)
  - The behaviour of the **whole** application becomes undefined

Atomic Operations in C++

- What we previously called “synchronizing operations” are called “atomic operations” in C++
  - As the name implies, these operations can be used to **atomically act on memory** – atomically read, write, update memory
  - Atomic means indivisible, whole operation appears to be performed without interruption, all threads see the operation **fully done or not done at all**
  - **But**: atomic operations have another role too, they are used for **synchronization between threads** (more in part 2)
  - As such, they are the **basis for all other sync. mechanisms** (condition variables, futures, mutexes, barriers, …)

- In C++, atomic operations are operations on **atomic types**

- Three classes of atomic types:
  - Basic **atomic boolean flag**: **std::atomic_flag**
  - **Standard atomic types**, e.g., **std::atomic<int>**
  - **User-defined atomic types**, e.g., **std::atomic<mystruct>**

Atomic Types in C++

std::atomic_flag

- Guaranteed to be lock free
- No load(), store(), or exchange() operations
- clear() and test_and_set() are available

Standard Atomic Types (std::atomic<int> etc.)

- May be lock free
- load(), store(), exchange()
- compare_exchange_weak(), compare_exchange_strong()
- fetch_and_OP(), as appropriate for the type
- operators, as appropriate for the type

User-defined Atomic Types

- Likely not lock free (may depend on size)
- load(), store(), exchange()
- compare_exchange_weak(), compare_exchange_strong()

std::atomic_flag

- The fundamental atomic data type in C++ is **std::atomic_flag**
  - **Guaranteed** to be **lock free** (unlike the other atomic data types)
  - Offers only very basic functionality, meant as a **building block**
  - Implemented using platform-dependent synchronization operations
  - Can be used to build more complex synchronization mechanisms, especially locks

- std::atomic_flag is an atomic boolean flag
  - Can be in one of two states: **set** and **clear**
  - Initially **always clear**, has to be initialized with ATOMIC_FLAG_INIT

- Only two member functions are available (before C++20)
  - **clear()** atomically clears the flag
  - **test_and_set()** atomically sets the flag and obtains its previous value
  - Both methods take an optional **memory ordering parameter** that defaults to **std::memory_order_seq_cst** (sequentially consistent)

Spinlock example using std::atomic_flag

```
#include <atomic>

class spinlock
{
   std::atomic_flag flag;

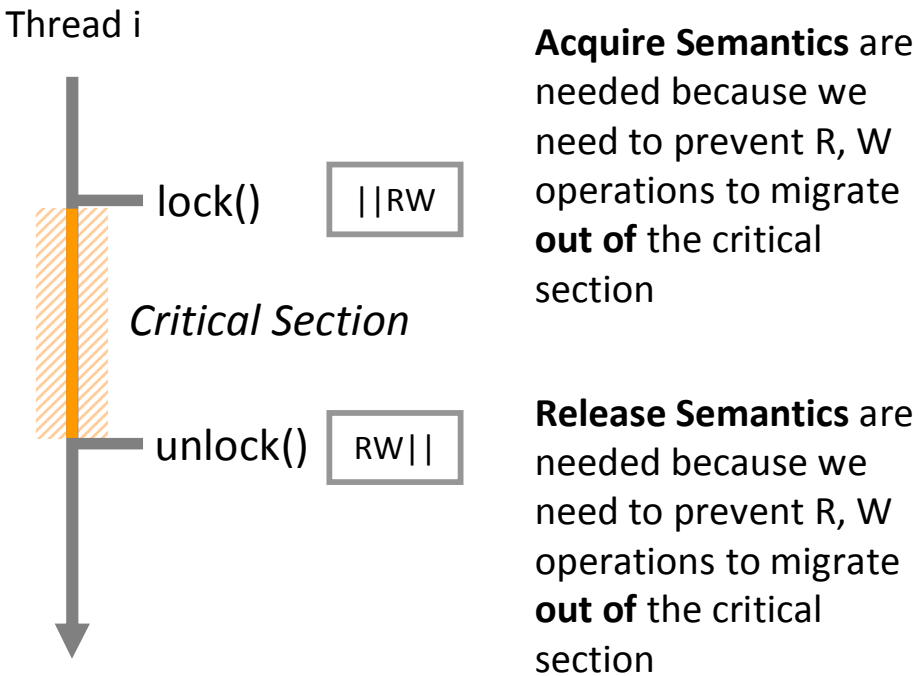
public:
   spinlock() :
      flag(ATOMIC_FLAG_INIT) {}

   void lock() {
      while(flag.test_and_set(
            std::memory_order_acquire))
         { /* spin */ }
   }

   void unlock() {
      flag.clear(
         std::memory_order_release);
   }
};
```

Optional memory ordering parameter, defaults to **std::memory_order_seq_cst**

- flag initialized in constructor
- unlock() simply clears the flag
- lock() spins until test_and_set() returns false
  - This indicates that this invocation changed the flag from *clear* to *set* and the lock was acquired



Acquire Semantics are needed because we need to prevent R, W operations to migrate **out of** the critical section

Release Semantics are needed because we need to prevent R, W operations to migrate **out of** the critical section

Memory Ordering Parameters

- Six different mem. ordering parameters, grouped into three models
  - Some do not apply to load or store operations
  - We'll discuss them in more detail in Part 2

	Load Operation	Store Operation	RMW Operation	
memory_order_seq_cst	x	x	x	sequentially consistent ordering
memory_order_relaxed	x	x	x	relaxed ordering
memory_order_acquire	x		x	acquire-release ordering
memory_order_release		x	x	
memory_order_acq_rel			x	
memory_order_consume	x		x	

Standard Atomic Types (1)

- For some built-in types there is a corresponding atomic type in C++
  - Applies to **boolean**, **integral**, and **pointer** types
  - These standard atomic types are realized as template specializations of the **std::atomic<T>** base template, e.g., **std::atomic<int> a;**
  - Also available under alternative names (typedefs), e.g., **atomic_bool b;**

- These atomic types **may be lock free**
  - Most platforms will be able to implement these types with support from hardware and won't need a lock
  - is_lock_free() member function can be used to find out at runtime
  - Macros can be used at compile time, e.g., ATOMIC_BOOL_LOCK_FREE

- ATOMIC_<TYPE>_LOCK_FREE may evaluate to
  - 0 The type is never lock free
  - 1 Depends – check at runtime with is_lock_free()
  - 2 The type is always lock free

Standard Atomic Types (2)

- SuperMUC-NG login node
  - g++ (GCC) 9.2.0
  - Intel(R) Xeon(R) Gold 6148 CPU @ 2.40GHz
  - Linux 4.4.180-94.97-default x86_64

```
ATOMIC_BOOL_LOCK_FREE     : 2
ATOMIC_CHAR_LOCK_FREE     : 2
ATOMIC_SHORT_LOCK_FREE    : 2
ATOMIC_INT_LOCK_FREE      : 2
ATOMIC_LONG_LOCK_FREE     : 2
ATOMIC_LLONG_LOCK_FREE    : 2
ATOMIC_CHAR16_T_LOCK_FREE: 2
ATOMIC_CHAR32_T_LOCK_FREE: 2
ATOMIC_WCHAR_T_LOCK_FREE  : 2
ATOMIC_POINTER_LOCK_FREE  : 2
```

- Nvidia Jetson Nano Board
  - g++ 7.4.0
  - ARMv8 Processor rev 1 (v8l)
  - Linux 4.9.140-tegra aarch64

```
ATOMIC_BOOL_LOCK_FREE     : 2
ATOMIC_CHAR_LOCK_FREE     : 2
ATOMIC_SHORT_LOCK_FREE    : 2
ATOMIC_INT_LOCK_FREE      : 2
ATOMIC_LONG_LOCK_FREE     : 2
ATOMIC_LLONG_LOCK_FREE    : 2
ATOMIC_CHAR16_T_LOCK_FREE: 2
ATOMIC_CHAR32_T_LOCK_FREE: 2
ATOMIC_WCHAR_T_LOCK_FREE  : 2
ATOMIC_POINTER_LOCK_FREE  : 2
```

Standard Atomic Types (3)

- Some usage restrictions on standard atomic types:
  - They don't support copy construction or copy-assignment
  - These operations involve two atomic objects and can't be made atomic as a whole without some extra effort

```
std::atomic<int> a, b;

a = b; // error – no copy assignment
std::atomic<int> c(b); // error – no copy constructor
```

- Conversion to and from the underlying types is available

```
std::atomic<int> a
int b;

a = b; // ok, assignment from int
std::atomic<int> c(b); // ok, construct from int
```

Load / Store / Exchange

- Atomic loads and stores can be performed with the **load**() and **store**() member functions
  - A memory ordering parameter can optionally be specified
  - Alternatively, the conversion / assignment operators can be used

```
std::atomic<int> val;

val = 42;
val.store(42); // same as above
val.store(42, memory_order_seq_cst); // same as above

int b = val;
b = val.load(); // same as above
b = val.load(memory_order_seq_cst); // same as above
```

- Exchange atomically stores the desired value and returns the previous one

```
int c = val.exchange(43);
c = val.exchange(43, memory_order_seq_cst) // same as above
```

Compare – Exchange (1)

- Like exchange, but conditional on the current value
  - Aka. compare-and-swap (CAS)
  - Two forms – "**weak**" and "**strong**"

```
bool compare_exchange_weak(T& expected, T desired,
                           std::memory_order success,
                           std::memory_order failure);

bool compare_exchange_strong(T& expected, T desired,
                             std::memory_order success,
                             std::memory_order failure);
```

- Compare Exchange
  - Boolean return value indicates whether exchange was performed
  - If not, upon return from the call, **expected is updated with the value found**
  - Two memory ordering parameters, one for the success case and one for the failure case

Compare – Exchange (2)

- Strong and weak forms
  - "**weak**" means that the operation can fail (return false) for **spurious** reasons, even though the object has the expected value; (it is "**unreliable**")
  - "**strong**" means that the operation will only fail if the object does not have the expected value; (it is "**reliable**")
- Some systems can natively offer only the weak form
  - The strong form can then be achieved simply by using the weak form in a loop
  - I.e., simply **retry** if expected is **unchanged**, but return value is **false**
- Sometimes compare-exchange is used in a loop in the application
  - So, there would be a potential double-loop that can be avoided by the application directly using the weak form in a loop

Example: Atomic Increment Operation

```
void atomic_incr(std::atomic<int>& val,
                 int incr)
{
  int oldval = val.load();
  int newval = oldval + incr;

  // bool compare_exchange_strong(T& expected, T desired);
  while( !val.compare_exchange_strong(oldval, newval) ) {
    oldval = val.load();  // redundant, because oldval is already
                          // updated by compare_exchange_strong
    newval = oldval + incr;
  }
}
```

- In this case, the fact that *expected* is passed by reference and updated by the call is desirable
  - In other cases it might be surprising or even unwanted
  - In that case, one can always make a copy and pass that to the call

Fetch and <OP> Operations

- The following read-modify-write operations are available, depending on the underlying type
  - **fetch_add** (arithmetic addition) – pointer types and integral types
  - **fetch_sub** (arithmetic subtraction) – pointer types and integral types
  - **fetch_and** (bitwise AND) – integral types only
  - **fetch_or** (bitwise OR) – integral types only
  - **fetch_xor** (bitwise XOR) – integral types only

- These operations
  - Atomically perform the specified operation and **return the original value**

Fetch and <OP> in the form of Operators

- Overloaded operators are available for the atomic types
  - These perform the equivalent Fetch and <OP> operation
  - +=, -=, ++, --, …
  - No memory order can be specified for the operator versions (defaults to memory_order_seq_cst)
  - Contrary to usual conventions, values and not references are returned

Operator	Equivalent to
`T operator+=(T arg)`	`fetch_add(arg) + arg`
`T operator-=(T arg)`	`fetch_sub(arg) - arg`
`T operator&=(T arg)`	`fetch_and(arg) & arg`
`T operator|=(T arg)`	`fetch_or(arg) | arg`
`T operator^=(T arg)`	`fetch_xor(arg) ^ arg`
`T operator++()  // pre-increment  ++val`	`fetch_add(1) + 1`
`T operator++(int) // post-increment val++`	`fetch_add(1)`
`T operator--()  // pre-decrement  --val`	`fetch_sub(1) - 1`
`T operator--(int) // post-decrement val--`	`fetch_sub(1)`

Fetch and <OP> Example

```
// provide a unique id for each invocation, by atomically
// increasing a counter, starting from 0
int next_id(std::atomic<int>& val)
{
  int id = val.fetch_add(1); // or simply id = val++;
  return id;
}
```

Used from OpenMP with multiple threads:

```
int main(int argc, char* argv[])
{
  std::atomic<int> ctr{0};

#pragma omp parallel
  {
    for( int i=0; i<10; i++ ) {
      cout << omp_get_thread_num()
           << " " << next_id(ctr) << "\n";
    }
  }
}
```

Output

```
0 0
0 2
…
0 9
0 10
1 1
1 11
..
1 19
```

Atomic User Defined Types

- std::atomic offers a base template that can turn a user-defined type (UDT) into an atomic type
  - E.g., std::atomic<UDT>
  - Implementation is likely based on a lock
  - If UDT is small (less then 8, 16 Bytes, implementation may be lock-free)

- Some restrictions apply
  - The type must have a trivial copy-assignment operator, no virtual functions
  - I.e., make a copy by invoking **memcpy**
  - In compare_exchange the comparison is performed using **memcmp**, not the comparison operator that may be defined for the type

Examples for Atomic User-Defined Types

```
struct mystruct {
 char a;
 int  b;
 float c;
};

std::atomic<mystruct> val; // fine!

std::atomic<std::vector<int>> a; // error
std::vector<std::atomic<int>> b; // fine!

std::atomic<double> c; // fine!
```

Non-member Functions

- The functionality described so far comes in the form of member functions of atomic types

- Most of this functionality is also available in the form of free-standing non-member functions
  - Those functions are meant to be C compatible
  - Pointers are used instead of references
  - Instead of function overloading, a different name is used for the versions allowing the ordering parameter ("_explicit") suffix

```
std::atomic<int> val;

val.store(42);
std::atomic_store(&val, 42); // same as above

val.store(32, std::memory_order_relaxed);
std::atomic_store_explicit(&val, 32,
                           std::memory_order_relaxed); // same as above
```

Atomic Types Summary (1)

- Summary from https://en.cppreference.com/

std::atomic

Member functions

(constructor)	constructs an atomic object (public member function)
`operator=`	stores a value into an atomic object (public member function)
`is_lock_free`	checks if the atomic object is lock-free (public member function)
`store`	atomically replaces the value of the atomic object with a non-atomic argument (public member function)
`load`	atomically obtains the value of the atomic object (public member function)
`operator T`	loads a value from an atomic object (public member function)
`exchange`	atomically replaces the value of the atomic object and obtains the value held previously (public member function)
`compare_exchange_weak` `compare_exchange_strong`	atomically compares the value of the atomic object with non-atomic argument and performs atomic exchange if equal or atomic load if not (public member function)

Atomic Types Summary (2)

- Summary from https://en.cppreference.com/

Specialized member functions

`fetch_add`	atomically adds the argument to the value stored in the atomic object and obtains the value held previously (public member function)
`fetch_sub`	atomically subtracts the argument from the value stored in the atomic object and obtains the value held previously (public member function)
`fetch_and`	atomically performs bit-wise AND between the argument and the value of the atomic object and obtains the value held previously (public member function)
`fetch_or`	atomically performs bit-wise OR between the argument and the value of the atomic object and obtains the value held previously (public member function)
`fetch_xor`	atomically performs bit-wise XOR between the argument and the value of the atomic object and obtains the value held previously (public member function)

`operator++`
`operator++(int)`
`operator--`
`operator--(int)`
	increments or decrements the atomic value by one
	(public member function)

`operator+=`
`operator-=`
`operator&=`
`operator|=`
`operator^=`
	adds, subtracts, or performs bit wise AND, OR, XOR with the atomic value
	(public member function)